Docker Beginner's Guide

A friendly, hands-on introduction to images, containers, ports, and Dockerfiles.

1. Core Concepts

Two words show up everywhere in Docker. Learn these first and the rest gets easier.

Term	What it is	Analogy
Docker image	A read-only (**immutable**) template that describes what a container contains.	A recipe
Docker container	A **running instance** of an image.	The meal cooked from the recipe

The basic workflow: **get an image, then run it.** Running an image creates a container.

> **Tip:** You can create many containers from one image, just like cooking the same recipe many times.

2. Getting Images

Pulling an image

```
# Download a specific version of nginx
docker pull nginx:1.23

# Download the hello-world test image
docker pull hello-world
```

Names and tags

An image is written as `name:tag`, where the **tag** is the version, for example `nginx:1.23` or `python:3.12-slim`.

> **Tip:** With no tag, Docker uses `latest`. Specify a version for predictable results.

Auto-pull

You don't have to pull first. If the image isn't on your computer, `docker run nginx` **downloads it automatically** and then runs it.

Listing your images

```
docker images                 # all downloaded images
docker image ls               # same, newer-style command
docker images | grep python   # filter the list
```

3. Running Containers

`docker run` creates and starts a container from an image.

```
# Run in the background (detached)
docker run -d nginx:1.23

# Run once and remove the container afterwards
docker run --rm hello-world

# Run a specific version
docker run python:3.12-slim

# Run one command, then clean up
docker run --rm python:3.12-slim python -c "print('Hello-world')"

# Open a shell inside Ubuntu (type exit to leave)
docker run -it ubuntu:latest bash
```

Useful options

Option	What it does
`-d`	**Detached mode.** Runs in the background so your terminal stays free.
`--rm`	**Auto-remove.** Deletes the container when it stops. Great for quick tests.
`-it`	**Interactive terminal.** Lets you type commands inside the container.
`-p host:container`	**Port binding.** Connects a port on your computer to one in the container.
`--name <name>`	**Custom name** instead of a random one.

> **Heads-up:** Each `docker run hello-world` without `--rm` creates a **brand-new container**. Old ones stay on your system, stopped, until you remove them.

4. Port Binding

Containers are isolated by default. To open a web server inside one from your browser, **bind a port**.

```
docker run -d -p 9000:80 nginx:1.23
```

Part	Meaning
`9000`	Port on **your computer**. Pick any free port.
`80`	Port **inside the container** where nginx listens.

Now open `http://localhost:9000` in your browser.

Important: The container port must match the port the app really listens on. nginx uses 80, so changing it breaks the connection. Only the host port is yours to choose.

5. Managing Containers

```
docker ps          # running containers
docker ps -a       # all containers, including stopped

# Give a container a name
docker run --name web-app -d -p 9000:80 nginx:1.23

# Read the logs (by ID or name)
docker logs 94e76d437e6a
docker logs web-app

docker stop <container>     # stop
docker start <container>    # resume where it left off
docker rm <container>       # remove a container
docker rmi <image>          # remove an image
```

Tip: Use the container **ID or name**. You only need the first few characters of an ID if they're unique.

Warning: You can't remove a running container, or an image a container is using. Stop and remove the container first.

6. Building Your Own Image

A **Dockerfile** is a text file of step-by-step instructions for building an image.

Rule: Every Dockerfile must begin with `FROM`.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt /app/
COPY src /app/

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 3000

CMD ["python", "server.py"]
```

```
docker build -t python-app:1.0 .
```

- `-t python-app:1.0` sets the image **name and tag**.
- The final `.` means "use the current folder as the build context".

> **Tip:** Images don't update themselves. After changing code, run `docker build` again, ideally with a new tag like `1.1`.

What each instruction does

`FROM python:3.12-slim`

Sets the **base image**. Docker downloads it from Docker Hub if it isn't already on your computer.

`WORKDIR /app`

Sets the **working directory** inside the image. Later steps run from here. Docker creates it if needed.

`COPY requirements.txt /app/`

Copies a file from your computer into the image, in the form `COPY <source> <destination>`.

- The source is relative to the **build context** (usually the Dockerfile's folder), not your whole computer.
- The destination is a folder inside the image, so the file lands at `/app/requirements.txt`.

`COPY src /app/`

Copies the contents of your `src` folder, subfolders included, into `/app/`. A `src/server.py` becomes `/app/server.py`.

`RUN pip install --no-cache-dir -r requirements.txt`

Installs your Python dependencies. `--no-cache-dir` keeps the image smaller.

`EXPOSE 3000`

Documents that the app listens on port **3000**.

> **Note:** `EXPOSE` is mostly documentation. To reach the app you still need `-p`, for example `docker run -p 3000:3000 python-app:1.0`.

`CMD ["python", "server.py"]`

The command that runs **when the container starts**.

RUN vs CMD

Instruction	When it runs	Typical use
`RUN`	At **build time**, while the image is created	Installing dependencies
`CMD`	At **container startup**	Launching your app

7. Putting It All Together

```
docker build -t python-app:1.0 .
docker run --name my-app -d -p 3000:3000 python-app:1.0
docker ps
docker logs my-app
docker stop my-app
docker rm my-app
```

You've covered the essentials: pulling images, running containers, binding ports, managing containers, and building your own image. Keep experimenting, because practice is the fastest way to make Docker feel natural.